```python
import keras
from keras import layers
from keras.datasets import mnist
import numpy as np
import matplotlib.pyplot as plt


# b. Access the dataset
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize data
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# Convert 28x28 images into 784 values
x_train = x_train.reshape((len(x_train), 784))
x_test = x_test.reshape((len(x_test), 784))

print("Training data:", x_train.shape)
print("Testing data :", x_test.shape)
```

```
Training data: (60000, 784)
Testing data : (10000, 784)
```

```python
# =========================================================
# c. Encoder - converts input into latent representation
# =========================================================
encoding_dim = 32
input_img = keras.Input(shape=(784,))
encoded = layers.Dense(
128, activation="relu"
)(input_img)

encoded = layers.Dense(
encoding_dim, activation="relu"
)(encoded)

# =========================================================
# d. Decoder - converts latent representation back
# to original input
# =========================================================
decoded = layers.Dense(
128, activation="relu"
)(encoded)
decoded = layers.Dense(
784, activation="sigmoid"
)(decoded)
# Create Autoencoder
autoencoder = keras.Model(input_img, decoded)
```

```python
# ============================================================
# e. Compile model
# ============================================================
autoencoder.compile(
optimizer="adam",
loss="binary_crossentropy",
metrics=["accuracy"]
)
autoencoder.summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native
Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU
will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
```

```
Model: "functional_1"
```

```
┏━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳━━━━━━━━━━━━━━━━━━━━━━━━━━━━━┳
━━━━━━━━━━━━━━━━━┓
┃ Layer (type)                         ┃ Output Shape                ┃
Param # ┃
┡━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇━━━━━━━━━━━━━━━━━━━━━━━━━━━━━╇
━━━━━━━━━━━━━━━━━┩
│ input_layer_1 (InputLayer)           │ (None, 784)                 │
0 │
├──────────────────────────────────────┼─────────────────────────────┼
─────────────────┤
│ dense_4 (Dense)                      │ (None, 128)                 │
100,480 │
├──────────────────────────────────────┼─────────────────────────────┼
─────────────────┤
│ dense_5 (Dense)                      │ (None, 32)                  │
4,128 │
├──────────────────────────────────────┼─────────────────────────────┼
─────────────────┤
│ dense_6 (Dense)                      │ (None, 128)                 │
4,224 │
├──────────────────────────────────────┼─────────────────────────────┼
─────────────────┤
│ dense_7 (Dense)                      │ (None, 784)                 │
101,136 │
└──────────────────────────────────────┴─────────────────────────────┴
─────────────────┘
```

```
 Total params: 209,968 (820.19 KB)

 Trainable params: 209,968 (820.19 KB)

 Non-trainable params: 0 (0.00 B)
```

```python
# ============================================================
# Train the Autoencoder
# =========================================================
autoencoder.fit(
x_train,
x_train,
epochs=10,
batch_size=256,
shuffle=True,
validation_data=(x_test, x_test)
)
```

```
Epoch 1/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 8s 22ms/step - accuracy: 0.0106 - loss:
0.2337 - val_accuracy: 0.0088 - val_loss: 0.1580
Epoch 2/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 23ms/step - accuracy: 0.0101 - loss:
0.1399 - val_accuracy: 0.0111 - val_loss: 0.1260
Epoch 3/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 10s 23ms/step - accuracy: 0.0105 - loss:
0.1202 - val_accuracy: 0.0093 - val_loss: 0.1144
Epoch 4/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 20ms/step - accuracy: 0.0101 - loss:
0.1112 - val_accuracy: 0.0108 - val_loss: 0.1060
Epoch 5/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 23ms/step - accuracy: 0.0104 - loss:
0.1049 - val_accuracy: 0.0122 - val_loss: 0.1011
Epoch 6/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 10s 22ms/step - accuracy: 0.0123 - loss:
0.1010 - val_accuracy: 0.0130 - val_loss: 0.0979
Epoch 7/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 5s 22ms/step - accuracy: 0.0135 - loss:
0.0982 - val_accuracy: 0.0142 - val_loss: 0.0961
Epoch 8/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 24ms/step - accuracy: 0.0134 - loss:
0.0963 - val_accuracy: 0.0129 - val_loss: 0.0942
Epoch 9/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 6s 24ms/step - accuracy: 0.0131 - loss:
0.0948 - val_accuracy: 0.0135 - val_loss: 0.0929
Epoch 10/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 4s 19ms/step - accuracy: 0.0129 - loss:
0.0935 - val_accuracy: 0.0123 - val_loss: 0.0919
```

```
<keras.src.callbacks.history.History at 0x137c4d38440>
```

```
# Anomaly Detection

import numpy as np
import matplotlib.pyplot as plt

# Reconstruct test images
```

```python
decoded_imgs = autoencoder.predict(x_test)

# Calculate reconstruction error
error = np.mean(
    np.square(x_test - decoded_imgs),
    axis=1
)

# Set threshold
threshold = np.percentile(error, 95)
print("Anomaly Threshold:", threshold)

# Detect anomalies
anomalies = error > threshold
print("Number of anomalies:", np.sum(anomalies))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
Anomaly Threshold: 0.020270457
Number of anomalies: 500
```

```python
# Display original and reconstructed images

n = 10
plt.figure(figsize=(20, 4))

for i in range(n):
    # Original image
    plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.title("Original")
    plt.axis("off")

    # Reconstructed image
    plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap="gray")
    plt.title("Reconstructed")
    plt.axis("off")

plt.show()


# Plot reconstruction error

plt.hist(error, bins=50)

plt.axvline(
    threshold,
    color="red",
    linestyle="--",
    label="Anomaly Threshold"
)

plt.xlabel("Reconstruction Error")
plt.ylabel("Number of Images")
plt.title("Anomaly Detection")
plt.legend()
plt.show()
```